

Item G — SOLID Mapping

Class Responsibilities

Every class in the system, with its single responsibility, before mapping how SOLID governs the relationships between them.

Class	Its ONE responsibility
Movie	Holds a movie's title, language and duration — nothing else.
Seat	Represents one physical seat: its seat number and its type (SILVER/GOLD/PLATINUM).
Screen	Represents one auditorium: its screen number, and it owns the fixed set of Seats inside it.
Cinema	Represents the theatre itself: its name, and it owns its Screens.
Show	Represents one screening — a Movie on a Screen at a specific time — and owns that show's ShowSeats.
ShowSeat	Tracks the AVAILABLE/BOOKED status of ONE seat FOR ONE specific show.
Customer	Holds a customer's name and phone number.
Booking	Holds a booking id, which show, which seats, the total amount, and the booking's status.
Payment (abstract)	Defines the payment contract only: pay(amount) — a pure virtual method, no implementation.
UpiPayment / CardPayment / CashPayment	Each implements HOW that one specific payment method actually pays.
PriceCalculator	Turns a list of booked seats into a total amount.
TicketPrinter	Formats and prints a ticket for a confirmed booking — printing only, no side effects.
BookingService	The orchestrator: runs the entire booking flow end to end, coordinating every other class.
MainMenu	Reads console input and displays the menu — holds no domain data of its own.

SOLID Principles — Statement, Justification, Application

S — Single Responsibility Principle

Principle: A class should have only one reason to change — one job, one axis of change.

Justification: If a class handles two unrelated jobs, a change to one job risks breaking the other, and two different people/reasons end up editing the same file for unrelated purposes. Separating responsibilities means each class can be understood, tested, and modified in isolation.

Application in this project: Booking does not print the ticket. It only stores booking data (id, show, seats, amount, status) and manages its own status transitions (confirm(), cancel(), fail()). Printing is a presentation concern, not a data concern — it belongs to TicketPrinter. If the ticket's printed layout changes (e.g. add a QR code line), only TicketPrinter is touched; if the pricing rule changes, only PriceCalculator is touched. Nobody has to edit Booking for either change. Similarly, Show knows and can print its own seat layout, but does not know how to calculate money, and BookingService orchestrates the flow but doesn't itself know how UPI payment works internally.

O — Open/Closed Principle

Principle: Software entities (classes, modules, functions) should be open for extension but closed for modification — new behaviour should be added, not patched into existing working code.

Justification: Every time an existing, tested class must be edited to support a new feature, there's a risk of breaking behaviour that already worked. If new behaviour can instead be added as a new class that plugs into an existing abstraction, the old code — and its test coverage — stays untouched and trustworthy.

Application in this project: Adding NetBanking as a new payment method requires writing exactly one new class, NetBankingPayment : public Payment, implementing pay(double amount).

BookingService::makePayment() already works against a Payment* (or a factory returning one), so no existing class is edited — BookingService, Booking, and PriceCalculator are all untouched. The system is open to this new payment behaviour, but closed to modification of the classes that already work.

L — Liskov Substitution Principle

Principle: Objects of a subclass must be substitutable for objects of the base class without altering the correctness of the program — a subclass must honour the exact contract of its base class, with no surprises.

Justification: If a caller has to know which concrete subclass it's holding (extra setup calls, special-casing, or a subclass that throws where the base type promised it wouldn't), polymorphism is broken and the caller can no longer treat all subclasses interchangeably — defeating the entire point of the base class.

Application in this project: Every subclass of Payment (UpiPayment, CardPayment, CashPayment) can be used wherever a Payment* (or Payment&) is expected, with no extra setup call and no surprise. BookingService only ever calls p->pay(amount) — it never checks typeid or downcasts to call a UPI-only method before paying. If a subclass required, say, upiPayment->setUpild() to be called first, or its pay() would throw unexpectedly, that subclass would violate LSP because it silently breaks the contract every other Payment honours.

I — Interface Segregation Principle

Principle: Clients should not be forced to depend on methods they do not use — prefer several small, specific interfaces over one large, general-purpose one.

Justification: A fat interface forces every implementer to either provide a meaningless/unsupported implementation of a method it doesn't need, or throw an exception at runtime — both are signs the interface should be split by what different clients actually need.

Application in this project: Payment is deliberately kept to a single method, pay(amount). We do not force a refund() method onto every Payment subclass, since (for example) CashPayment may not support programmatic refunds the way UpiPayment might. Forcing all payment types to implement refund() would make classes implement methods they can't meaningfully support — exactly what ISP warns against. If refunds are needed later, a separate Refundable interface would be introduced and only implemented by the payment types that actually support it.

D — Dependency Inversion Principle

Principle: High-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details — details should depend on abstractions.

Justification: If a high-level orchestrator (like a booking flow) is wired directly to one concrete low-level class, every new variant of that low-level class forces a change high up in the system — the exact coupling OCP and LSP are trying to avoid. Depending on an abstraction instead lets the high-level logic stay fixed while low-level implementations come and go.

Application in this project: BookingService does not construct a concrete CardPayment itself deep inside its booking logic and hard-wire itself to it. It receives/creates a Payment* behind the abstract Payment interface (selected by the customer's menu choice), and calls only pay() through that interface. BookingService depends on the abstraction Payment, not on any concrete payment class — so it never needs to change when a new concrete payment type is added (ties back into O).